

## **Unit – 4 : Session – 1 : SLO – 2 | Functional vs Imperative Paradigms**

### **I. Discussion Question**

1. A pure functional approach is better for adding two numbers because the operation is a pure function: given the same two inputs it always returns the same result, with no dependence on — and no modification of — any external state.
2. Predictability: `add(2, 3)` is always 5, regardless of when or how many times it is called, whereas an imperative version may read or mutate global variables, so results can change with program state.
3. Testability: pure functions need no setup or mocks — they can be unit-tested in isolation.
4. Referential transparency: any call can be replaced by its result, enabling safe refactoring, caching (memoization) and parallelism.
5. No side effects: the calculator's other features can never be accidentally corrupted by the addition logic, which is a real risk with imperative state mutation.